Johannes Husevåg Standal og Rozalia Anna Ciupka, 22.10.2026

CA3

Imports

In [134…
```
import sklearn as sk
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt

# progress bar for model training
import tqdm
```

Reading data

In [74]:
```
df_train = pd.read_csv("train.csv")
df_kaggle = pd.read_csv("test.csv")

df_train.shape
```

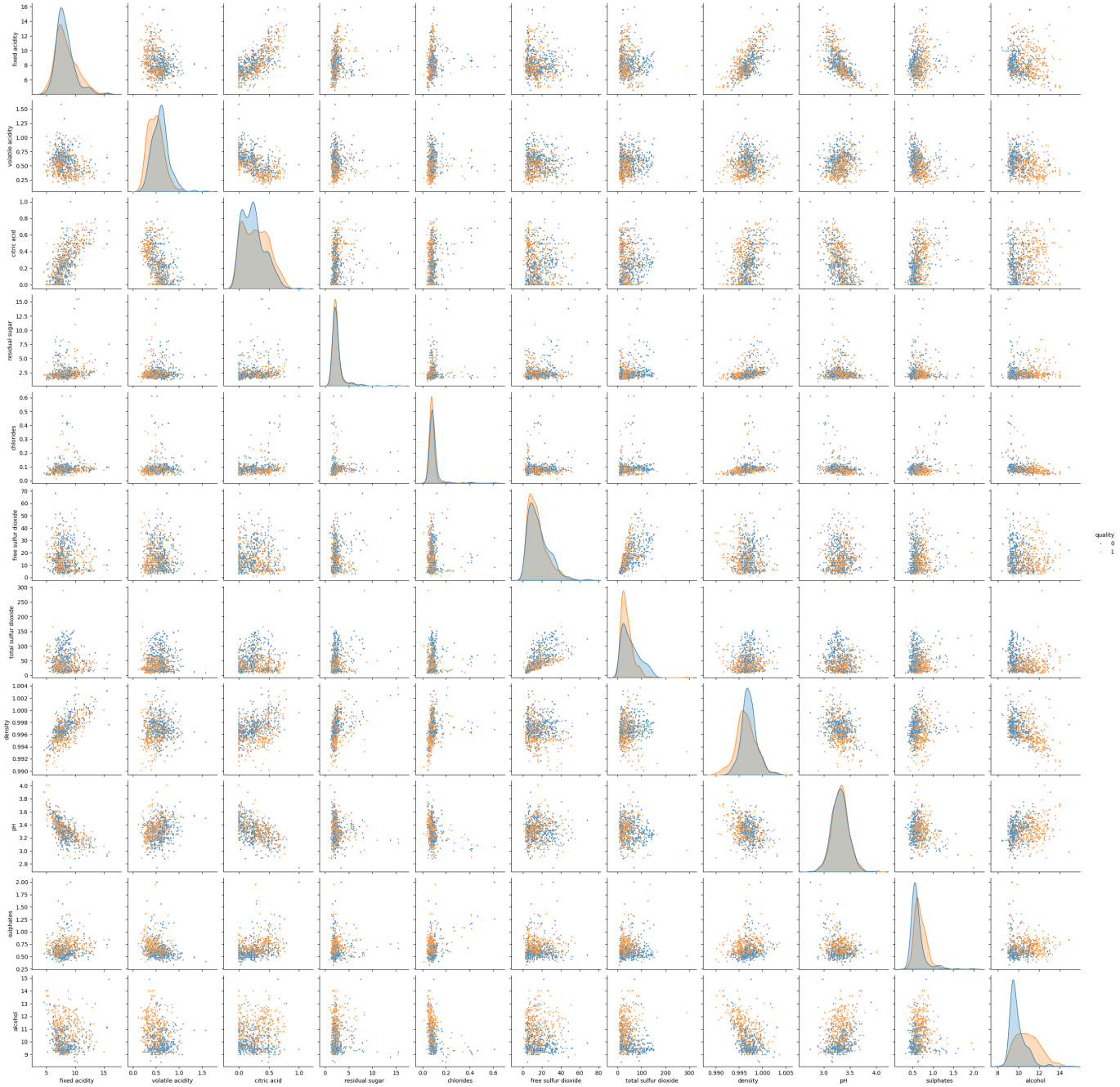
Out[74]:
```
(835, 13)
```

Data exploration and visualisation

In [75]:
```
df_visual = df_train.drop('Id', axis=1)

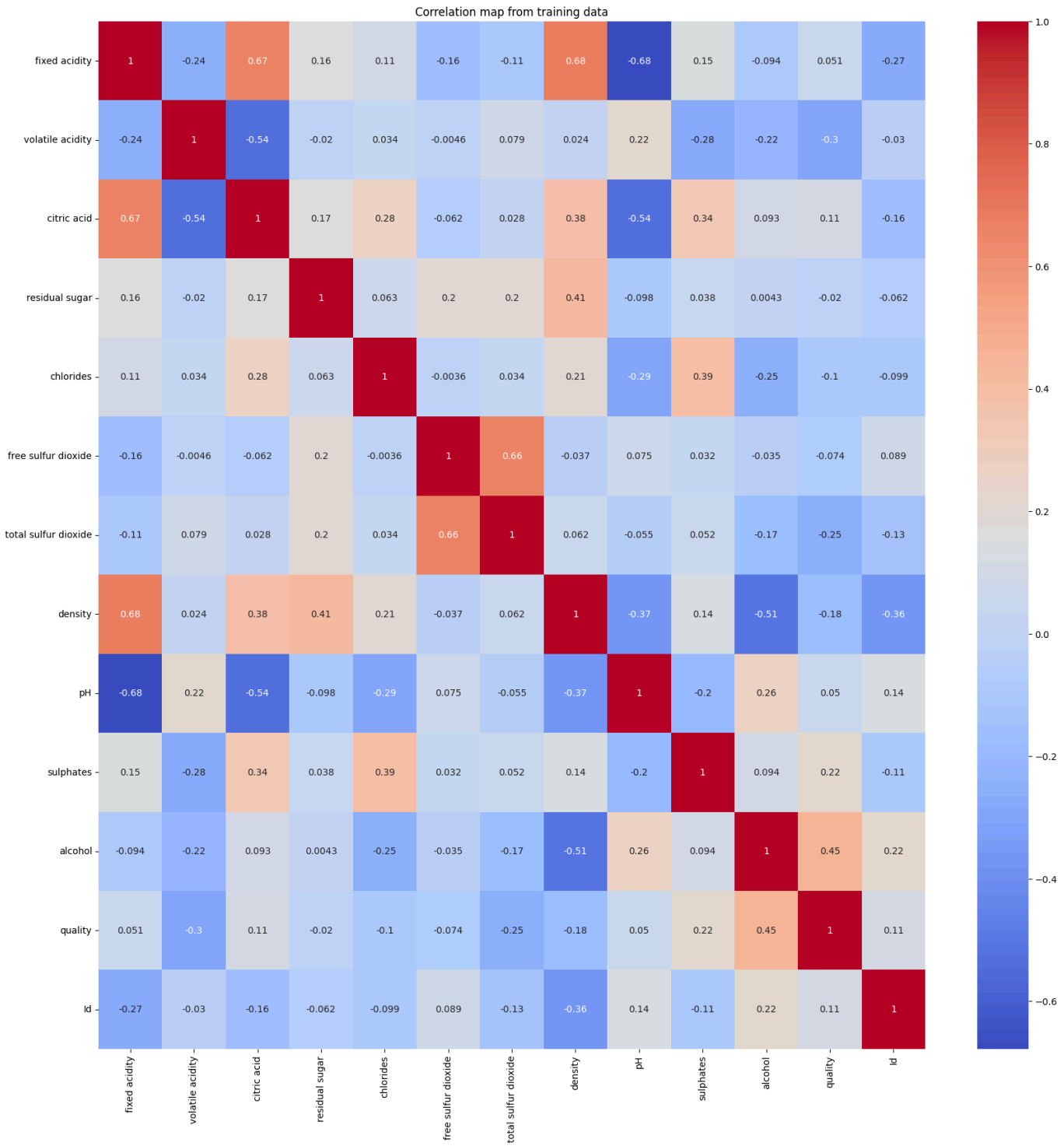
sns.pairplot(df_visual,  hue='quality', plot_kws={'s':5})
```

Out[75]:
```
<seaborn.axisgrid.PairGrid at 0x20811f90400>
```



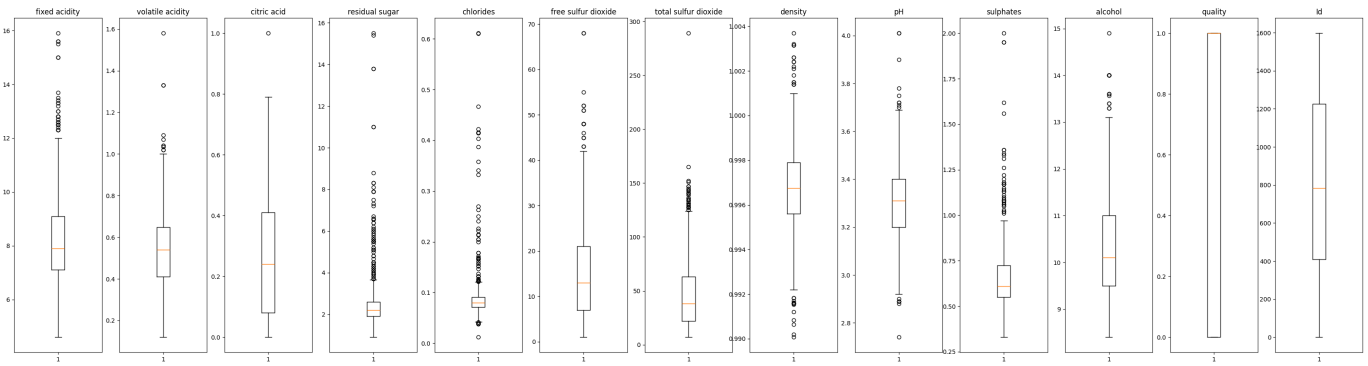
In [ ]:
```
# correlation map code from lecture 4
plt.figure(figsize=(20,20))
sns.heatmap(df_train.corr(), annot=True, cmap='coolwarm')
plt.title("Correlation map from training data")
```

Out[ ]:
```
Text(0.5, 1.0, 'Correlation map from training data')
```



In [ ]:
```
# whisker plots
fig, ax = plt.subplots(1, len(df_train.columns), figsize=(40,10))

for i, column in enumerate(df_train.columns):
    ax[i].set_title(column)
    ax[i].boxplot(df_train[column])
```



Data cleaning

Inspecting the plots.

Pairplot: Not many features are very clearly linearly seperable, so there are no clearly decisive features to use for the model. There are also no features that are clearly non-linearly seperable using the kernel trick. The thing we can see is that both chlorides and residual sugar have an almost linear destribution that can warant dropping these features.

BoxPlot: The values in the dataset have spread values as well as outliers, and will therefore benefit from scaling

Correlation map: There is no feature that has a high enough correlation value to another feature, that justifies removing a feature

In [8]:
```
df_train = df_train.dropna(axis=0)
```

Data preprocessing and visualisation

In [44]:
```
x_train = df_train.drop(['quality', 'Id', 'chlorides', 'residual sugar'], axis=1)
y_train = df_train['quality']

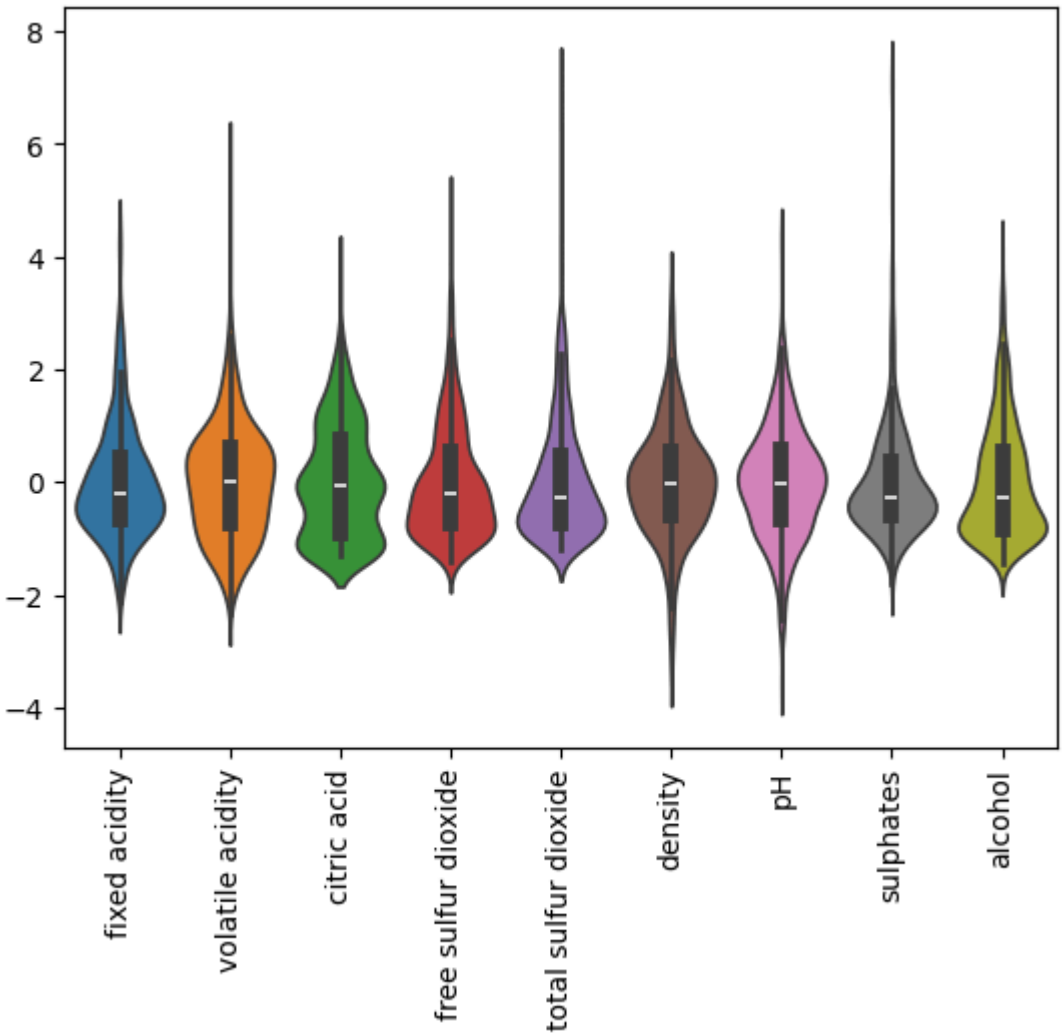
X_train, X_test, Y_train, Y_test = sk.model_selection.train_test_split(
    x_train, y_train,
    test_size=0.2,
    random_state=42,
    stratify = y_train
)

scaler = sk.preprocessing.StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

X_train_scaled_df = pd.DataFrame(X_train_scaled, columns=x_train.columns)
sns.violinplot(X_train_scaled_df)
plt.xticks(rotation=90)
```

Out[44]:
```
([0, 1, 2, 3, 4, 5, 6, 7, 8],
 [Text(0, 0, 'fixed acidity'),
  Text(1, 0, 'volatile acidity'),
  Text(2, 0, 'citric acid'),
  Text(3, 0, 'free sulfur dioxide'),
  Text(4, 0, 'total sulfur dioxide'),
  Text(5, 0, 'density'),
  Text(6, 0, 'pH'),
  Text(7, 0, 'sulphates'),
  Text(8, 0, 'alcohol')])
```



The data is scaled, and the outliers were left to be as they were.

Modelling

In [42]:
```
num_seeds = 100
models = []

# Logistic Regression
for c in [0.01, 0.1, 1, 10, 100]:
    models.append(("Logistic", {"C": c, "max_iter": 2000}))


# Support Vector Machine
for c in [0.1, 1.0, 10, 100]:
    for gamma in [0.001, 0.01, 0.1, 1]:
        models.append(("SVC", {"C": c, "gamma": gamma, "kernel": "rbf"}))


# Decision Tree
for criterion in ["gini", "entropy"]:
    for max_depth in [2, 3, 4, 5, 6, 8, 10, None]:
        for samples_split in [2, 5, 10]:
            models.append(("Tree", {"criterion": criterion, "max_depth": max_depth, "min_samp


# Random Forest
for criterion in ["gini", "entropy"]:
    for max_depth in [4, 6, 10, None]:
        for n_estimators in [100, 300]:
            models.append(("Forest", {"criterion": criterion, "max_depth": max_depth, "n_esti

# K-Nearest Neighbors
```

```python
for n_neighbors in [3, 5, 7, 9, 11, 15, 20, 25, 30]:
    for p in [1, 2]:
        models.append(("KNN", {"n_neighbors": n_neighbors, "p": p}))

def trainModels(x_train, y_train):
    print(f"Training {len(models)} model variatons with {num_seeds} different seeds")

    results = {
        "Model_index": [],
        "Name": [],
        "Accuracy": [],
        "Seed": []
    }

    pbar = tqdm.tqdm(total= len(models) * num_seeds)

    for seed in range(0, num_seeds):
        X_train, X_test, Y_train, Y_test = sk.model_selection.train_test_split(
            x_train, y_train,
            test_size=0.2,
            random_state=seed,
            stratify = y_train
        )

        scaler = sk.preprocessing.StandardScaler()

        X_train_scaled = scaler.fit_transform(X_train)
        X_test_scaled = scaler.transform(X_test)

        for model_index, (modelName, params) in enumerate(models):
            # Set up model
            if modelName == "Logistic":
                model = sk.linear_model.LogisticRegression(**params, random_state=seed)
                model.fit(X_train_scaled, Y_train)
                y_pred = model.predict(X_test_scaled)

            elif modelName == "SVC":
                model = sk.svm.SVC(**params, random_state=seed)
                model.fit(X_train_scaled, Y_train)
                y_pred = model.predict(X_test_scaled)

            elif modelName == "Tree":
                model = sk.tree.DecisionTreeClassifier(**params, random_state=seed)
                model.fit(X_train, Y_train)
                y_pred = model.predict(X_test)

            elif modelName == "Forest":
                model = sk.ensemble.RandomForestClassifier(**params, random_state=seed)
                model.fit(X_train, Y_train)
                y_pred = model.predict(X_test)

            elif modelName == "KNN":
                model = sk.neighbors.KNeighborsClassifier(**params)
                model.fit(X_train_scaled, Y_train)
                y_pred = model.predict(X_test_scaled)

            # Test model accuracy on test data
            accuracy = sk.metrics.accuracy_score(Y_test, y_pred)

            results['Model_index'].append(model_index)
            results["Name"].append(modelName)
            results["Accuracy"].append(accuracy)
            results["Seed"].append(seed)

            pbar.update(1)
```

```
    pbar.close()

    results_df = pd.DataFrame(results)

    results_mean_df = results_df.groupby(["Model_index", "Name"])['Accuracy'].mean().reset_in
    results_mean_df = results_mean_df.sort_values(by='Accuracy', ascending=False)

    return results_mean_df

result = trainModels(x_train, y_train)
print(result.head())
```

```
Training 103 model variatons with 100 different seeds
100%|██████████| 10300/10300 [05:36<00:00, 30.59it/s]
    Model_index    Name  Accuracy
84           84  Forest  0.769401
82           82  Forest  0.768383
83           83  Forest  0.766467
74           74  Forest  0.766467
81           81  Forest  0.766287
```

Final evaluation

In [43]:
```
best_model = result.iloc[0]

best_index = int(best_model["Model_index"])
modelName = best_model["Name"]
accuracy = best_model["Accuracy"]
best_parameters = models[best_index][1]

print(f"Best model was {modelName} with an mean accuraccy of {(100 * accuracy):.2f}%")
print("Here are the parameters used:")
print(best_parameters)
```

```
Best model was Forest with an mean accuraccy of 76.94%
Here are the parameters used:
{'criterion': 'entropy', 'max_depth': None, 'n_estimators': 300}
```

In [118…
```
# Set up the final model on all of the training data
X_train = x_train.to_numpy()
Y_train = y_train.to_numpy()

# Scale training data
scaler = sk.preprocessing.StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

if modelName == "Logistic":
    final_model = sk.linear_model.LogisticRegression(**best_parameters, random_state=42)
    final_model.fit(X_train_scaled, Y_train)

elif modelName == "SVC":
    final_model = sk.svm.SVC(**best_parameters, random_state=42)
    final_model.fit(X_train_scaled, Y_train)

elif modelName == "Tree":
    final_model = sk.tree.DecisionTreeClassifier(**best_parameters, random_state=42)
    final_model.fit(X_train, Y_train)

elif modelName == "Forest":
    final_model = sk.ensemble.RandomForestClassifier(**best_parameters, random_state=42)
    final_model.fit(X_train, Y_train)

elif modelName == "KNN":
    final_model = sk.neighbors.KNeighborsClassifier(**best_parameters)
```

```
    final_model.fit(X_train_scaled, Y_train)

print(f"Model trained succesfully")
```

```
Model trained succesfully
```

Kaggle submission

In [119…

```
# Sperate data
x_kaggle = df_kaggle.drop(['Id', 'chlorides', 'residual sugar'], axis=1)

if modelName in ["Logistic", "SVC", "KNN"]:
    x_input = scaler.transform(x_kaggle)
else:
    x_input = x_kaggle

# Predict targets and create a dataframe and CSV file
y_test = final_model.predict(x_input)
y_test = pd.DataFrame(y_test, columns=["quality"])
y_test.index.name = "Id"
y_test[['quality']].to_csv("submission.csv")
```

```
c:\Users\johan\GitHub\DAT200\.venv\lib\site-packages\sklearn\utils\validation.py:2742: UserWar
ning: X has feature names, but RandomForestClassifier was fitted without feature names
  warnings.warn(
```